Mime: Universal Speech, Visualized in Real-Time

COMP4107 Project Report

Winter 2026 - April 10, 2026
Carleton University

Group 10 - Daniel Lu, Methira Herath
101304181, 101304349
daniellu@cmail.carleton.ca, methiraherath@cmail.carleton.ca

Instructor: Matthew Holden

Table of Contents

Statement of Contributions

This project was completed by Daniel Lu and Methira Herath (Group 10). Both members made significant and equal contributions to nearly all aspects of the project.

Daniel was responsible for the development of the audio-to-blendshape neural model, including the architecture design, training loop implementation, and real-time inference optimization. He also was the primary contributor to the integration of the speech translation pipeline and the Zoom bridge component that allows for Mime to be run in real-time Zoom meetings.

Methira was responsible for the development of the blendshape rendering system, including the avatar rigging, mouth overlay rendering pipeline, and virtual camera integration. He also contributed significantly to model training and evaluation, as well as critical debugging and optimization work across the entire codebase.

Chapter 1

Introduction

Language barriers in video communication are no longer only a text or audio problem. Recent progress in speech-to-text, machine translation, and text-to-speech has made multilingual conversations more accessible, but live conversations over video still feel unnatural when the spoken audio and visible mouth motion do not match. This mismatch introduces an *immersion gap*: listeners hear translated speech while seeing lip movements that correspond to a different language and timing. In practical settings such as online meetings, this visual-audio inconsistency can reduce comprehension, increase cognitive load, and make interactions feel less human.

This report presents **Mime**, a real-time multimodal communication system that chains speech translation with synchronized facial animation, running on consumer hardware with latency low enough for live calls. Rather than generating synthetic face frames from scratch, Mime predicts ARKit facial blendshape coefficients directly from audio features and renders them on an animated avatar: an approach that intentionally trades some visual fidelity for the speed that *live operation* requires.

1.1 Background

Human communication is strongly multimodal. Timing, facial movement, and visual articulation all contribute to how spoken content is perceived, and disruptions to that coherence are not merely aesthetic - they measurably affect comprehension and naturalness [1]. In cross-lingual video calls, even accurate translation leaves this visual layer broken: the listener hears one language while watching lip movements shaped by another.

State-of-the-art visual dubbing systems address this problem, but they were designed for film post-production. They assume offline workflows, high-end compute, and closed pipelines - none of which are compatible with live telepresence. Mime targets the practical gap: continuous microphone capture, real-time translation, audio-driven facial animation, and direct integration with standard conferencing stacks on Windows.

1.2 Motivation and Objectives

The project is motivated by a simple goal: make multilingual video conversation feel coherent in both sound and appearance. Instead of treating translation and lip-sync as separate tools, Mime integrates them into one stream-oriented system. The platform is designed around three core objectives:

1. **End-to-end integration:** Build a real-time pipeline that chains speech recognition, machine translation, and synthesized speech with an audio-to-blendshape visual model. This project will focus on specifically **English-to-French translation**.
2. **Temporal visual coherence:** Produce stable lip-sync by mapping speech features to 52 ARKit facial blendshape coefficients and rendering an animated avatar suitable for live mouth overlay on a virtual camera output.
3. **Deployment practicality:** Maintain low latency on consumer-grade hardware, with integration support for common video conferencing workflows. This project will target **Zoom** as the primary use case, but the architecture should be adaptable to other platforms with the use of virtual camera and audio interfaces.

In implementation terms, Mime uses modular processors for STT, MT, and TTS, then routes synthesized audio to both playback and a fast inference path for facial animation. This architecture enables synchronized media output and reduces duplicated processing in live operation.

1.3 Related Prior Work

The main tools people have built for this are visual dubbing systems. Wav2Lip [2] masks the lower face of a video and inpaints it conditioned on audio, achieving lip sync close to real recordings. VideoReTalking [3] adds expression normalization and face enhancement on top of a similar approach, producing cleaner results. SadTalker [4] goes further, predicting 3D morphable model coefficients from audio to get more natural head movement than purely pixel-level methods. All three produce good output, but they assume pre-recorded video and offline compute. None are designed for a live call.

The parametric side of the literature is more relevant to Mime's approach. VOCA [5] showed you can map audio features directly to a structured face representation, specifically 3D mesh vertex displacements, and animate a wide range of identities from a single model. Mime follows the same principle but targets ARKit blendshape coefficients, which are lighter to infer and render directly on avatar-based output. For the speech side, Whisper [6] provides a practical multilingual transcription foundation, trained on 680,000 hours of audio across dozens of languages with near-human accuracy in a zero-shot setting.

The gap none of these works address is the *full pipeline*: translation, synthesis, and synchronized facial animation running together in real time. That is the specific scope Mime targets.

Chapter 2

Methods

2.1 Methods from Neural Networks

Mime is implemented as two coupled neural pipelines: a **speech translation pipeline** (STT → MT → TTS) and an **audio-to-blendshape (ABS) pipeline** that drives a 3D ARKit avatar in real time.

2.1.1 Speech Translation Pipeline

Speech-to-Text (STT): The microphone stream is captured at 16 kHz mono and segmented online using an RMS-based voice activity strategy. Chunks are finalized after a silence window and transcribed with Groq's `whisper-large-v3-turbo`. This chunked design reduces unnecessary API calls and keeps inference focused on complete utterances, rather than segments of non-contextual speech.

Machine Translation (MT): Each English utterance is translated to conversational French using `llama-3.3-70b-versatile` through a constrained system prompt. A short rolling context buffer is included to improve coherence for fragments and pronoun resolution during live dialogue.

Text-to-Speech (TTS): French text is synthesized by Inworld TTS (default model `inworld-tts-1.5-mini`, French voice). Audio is streamed as LINEAR16[7] at 48 kHz so playback can begin before sentence completion, which lowers perceived latency in meetings.

2.1.2 Audio-to-Blendshape Neural Model

The ABS model maps raw 16 kHz mono audio to 52 ARKit blendshape coefficients at 60 FPS. It follows an encode-then-predict structure: a four-layer progressive 1D convolutional backbone extracts hierarchical phonetic features with increasing channel depth, a Transformer encoder models long-range co-articulation dependencies, and a two-layer regression head with a final sigmoid activation constrains outputs to $[0, 1]$. The model supports two backbone modes: a custom CNN (Mode B, used in all experiments) and an optional pretrained Wav2Vec2 encoder [8] (Mode A), included for future comparison. Table 2.1 summarises the custom CNN configuration.

Loss. The primary objective is mean squared error over predicted and target blendshape sequences:

$$\mathcal{L}_{\text{mse}} = \frac{1}{N} \sum_{t} \|\hat{Y}_t - Y_t\|_2^2$$

Table 2.1: ABS architecture (H=512, L=4, heads= 8).

Layer	Key Attributes	Out Dim	Params
Conv1 + BN + ReLU	C=64, kernel=10, stride=4	$T/4 \times 64$	0.8 K
Conv2 + ReLU	C=128, kernel=4, stride=4	$T/16 \times 128$	32.9 K
Conv3 + ReLU	C=256, kernel=4, stride=4	$T/64 \times 256$	131.3 K
Conv4 + ReLU	C=H, kernel=4, stride=4	$T/256 \times H$	524.8 K
Transformer Encoder	L layers, d=H, heads=8, FFN dim= 2048	$T/256 \times H$	12.6 M
Regressor Head	Linear(H→256) + LeakyReLU(0.2) + Linear(256→52) + Sigmoid	$T/256 \times 52$	144.7 K
Total			**13.4 M**

Early experiments produced jittery animation, so a velocity consistency term was added to penalise erratic frame-to-frame changes:

$$\mathcal{L} = \mathcal{L}_{\text{mse}} + \lambda\,\mathcal{L}_{\text{vel}}, \quad \mathcal{L}_{\text{vel}} = \frac{1}{N'}\sum_{t}\|(\hat{Y}_t - \hat{Y}_{t-1}) - (Y_t - Y_{t-1})\|_2^2$$

λ=0.5 was fixed after informal testing balanced smoothness against per-frame MSE. Training uses the Adam optimizer with gradient clipping (max norm = 1.0).

Hyperparameter Search. A grid search was run over the parameters most likely to affect capacity and stability, shown in Table 2.2. The best configuration was selected by lowest validation MSE; H=512 consistently outperformed H=256, and longer training (20 epochs) improved convergence where early stopping was not triggered.

Table 2.2: Hyperparameter search space and selected values.

Hyperparameter	Values Tried	Best
Learning rate	1×10^{-4}, 5×10^{-5}	1×10^{-4}
Hidden dim H	256, 512	512
Batch size	4	4
Epochs	5, 20	20
Pretrained backbone	False	False

2.1.3 Real-Time Inference and Rendering

At runtime, the ABS model runs on a sliding audio window (0.4 s at 48 kHz) with queue-based frame updates. Predicted blendshapes are remapped from BEAT ordering to canonical ARKit ordering and applied to morph targets in a rigged GLB avatar via an offscreen renderer. A transport bridge publishes rendered frames to a virtual camera and TTS audio to a virtual cable device, enabling direct integration with Zoom-compatible workflows.

2.2 Dataset and Data Processing

The model is trained on the **BEAT multimodal dataset** [9] (ECCV 2022), a 76-hour motion-capture corpus from 30 speakers annotated with synchronized audio, text, emotion labels, and *52-dimensional facial blendshape weights* - matching the ARKit target space exactly. The English subset (`beat_english_v0.2.1`, 60 h) is used, though additional languages available in BEAT could be explored in future work.

Each sample pairs a `.wav` and `.json` file: audio is loaded as mono at 16 kHz and clipped to 10 seconds, with blendshape targets aligned at 60 FPS consistent with BEAT's native capture rate. When a clip is truncated, the target frame count is scaled proportionally to maintain audio-target alignment. Variable-length sequences are padded per batch. An 80/20 train/validation split with a fixed random seed ensures reproducibility.

2.3 Validation Strategy and Experiments

2.3.1 Model-Level Validation

Validation runs every epoch using MSE and MAE over the held-out 20% split. TensorBoard logs train and validation loss curves alongside MAE per epoch, and best checkpoints are saved by lowest validation MSE. To assess the contribution of the velocity consistency term, models trained with $\lambda = 0.5$ are compared against MSE-only runs ($\lambda = 0$), with MAE used to measure any improvement in per-coefficient smoothness.

2.3.2 Hyperparameter Experiments

Grid search trials are run over learning rate, hidden dimension, and epoch budget (Table 2.2). Each trial writes isolated TensorBoard logs and saves the best checkpoint for direct comparison of convergence behavior and final validation performance.

2.3.3 Iterative Model Development

Live testing of the initial model revealed a consistent temporal synchronisation failure: predicted blendshapes would rush ahead of the speech signal, producing visibly misaligned lip motion. This was attributed to the absence of positional encoding and no explicit residual temporal mixing, limiting the model's ability to resolve fine-grained frame ordering. Late in the project, a revised architecture was trained and found to exceed the lip-sync performance of the original. Key changes include: a three-layer convolutional backbone (strides 2-2-2, SiLU activations, BatchNorm throughout); a depthwise-separable **temporal mixer** with a residual connection; **sinusoidal positional encoding** with pre-Transformer LayerNorm; and a Transformer encoder with `norm_first=True` and GELU activation for improved training stability.

2.3.4 System-Level Validation

The full pipeline is validated in end-to-end live runs: microphone speech → transcription → translation → French synthesis → synchronized avatar mouth overlay via Zoom bridge. Concrete pass criteria are defined: first-audio-chunk latency below 800 ms; translation quality verified by manual review of 20 sampled utterances; and lip-sync coherence maintained with no perceptible frame drops over a sustained 5-minute session. Robustness under device reconnect and stream interruption is additionally assessed from operational logs.

Chapter 3

Results

3.1 Quantitative Data

All of the following results can be found in the results folder of the project GitHub repository, if too small to view here: https://github.com/2026W-COMP4107/mime/tree/main/results.

3.2 Qualitative Results

Chapter 4

Discussion

4.1 Limitations and Challenges

Tobio's development came with its fair share of challenges. The most significant was achieving an accurate court segmentation system, finding a method for consistent player identification, and reliable event detection.

While the player tracking model is good at finding players (mAP50 of 0.94), the bounding boxes aren't always precise (mAP50-95 of 0.61) and the model often swapped player IDs. In the chaos of a real game, similar-looking players sometimes led to devastating identity switches, and would have to be manually corrected in post.

Moreover, the models are also only as good as their training data. The court segmentation dataset, for example, was fairly clean and didn't include the messy, overlapping lines of multi-use gyms where many amateur teams play. This means performance could dip in those more common, imperfect environments.

In addition, compiling the 3D spatial coordinates of the ball was one of the biggest initial constraints, since our objective was to use a single camera viewpoint. Though exact XYZ could be obtained, the depth information was unreliabile without more powerful models and more research.

The most difficult challenge had to be event detection. Volleyball actions are continuous and fluid, making it hard to define clear start and end points for each event, especially in a sport as complicated and fast-paced as volleyball. The model often confused similar actions, like spikes and serves, leading to misclassifications that would need manual review. The overall system I developed was only a prototype and had some questionable logic and reliability issues due to it's complexity, but I know that a well-polished system is definitely within arms reach with more time and resources.

4.2 Directions for Future Work

I definitely see a lot of potential in improving Tobio further, both in terms of turning it into a fully-polished product by improving the technical components, as well as expanding into other sports and use-cases. First of all, I'd like to improve the recall of the most important subcomponents: ball tracking, player recognition, and event detection. These form the backbone of what's possible with Tobio, so improving their reliability is a priority. Additionally, I'd like to improve my application's practicality with real competitive teams' use-cases in mind. This includes building more meaningful visualizations, more powerful and flexible user interface that allows for each teams' customizations, and curated reports for both coaches and players.

Although, other more popular sports already have implementations of similar systems, these popular sports often have paid subscriptions to such services and the non-popular sports don't receive enough attention for innovation. Volleyball is just the start line, and I'd like to examine what possiblities exist in other domains.

4.3 Implications and Impact

I believe that Tobio has the potential to democratize sports performance analysis, since one of my key morals is making it accessible to anyone, including teams that lack the resources of professional organizations. Through automating the entire process of video analysis, teams can save opportunity costs on tedious manual data collection and focus more on the other significant aspects one would much rather be doing.

4.4 Conclusions

In conclusion, this project has successfully developed a proof-of-concept AI-powered platform for volleyball video analysis. While there are still many shortcomings due to the limited scope and time constraints of this project, Tobio's end-to-end system is still able to show a glimpse into what is possible when combining computer vision, machine learning, and sports analytics. With proper development and time, Tobio has the potential to become a valuable tool for amateur and collegiate volleyball teams, all while paving the way for similar systems in other sports.

References

[1] H. McGurk and J. MacDonald, "Hearing lips and seeing voices," *Nature*, vol. 264, no. 5588, pp. 746–748, 1976.

[2] K. R. Prajwal, R. Mukhopadhyay, V. P. Namboodiri, and C. Jawahar, "A lip sync expert is all you need for speech to lip generation in the wild," in *Proceedings of the 28th ACM International Conference on Multimedia*, MM '20, (New York, NY, USA), pp. 484–492, Association for Computing Machinery, 2020.

[3] K. Cheng, X. Cun, Y. Zhang, M. Xia, F. Yin, M. Zhu, X. Wang, J. Wang, and N. Wang, "Videoretalking: Audio-based lip synchronization for talking head video editing in the wild," 2022.

[4] W. Zhang, X. Cun, X. Wang, Y. Zhang, X. Shen, Y. Guo, Y. Shan, and F. Wang, "Sadtalker: Learning realistic 3D motion coefficients for stylized audio-driven single image talking face animation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8652–8661, 2023.

[5] D. Cudeiro, T. Bolkart, C. Laidlaw, A. Ranjan, and M. J. Black, "Capture, learning, and synthesis of 3D speaking styles," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10101–10111, 2019.

[6] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, pp. 28492–28518, PMLR, 2023.

[7] Google Cloud, "Linear16: Audio encoding for speech-to-text." https://docs.cloud.google.com/speech-to-text/docs/encoding#uncompressed_audio, 2026. Accessed: 2026-04-10.

[8] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," 2020.

[9] H. Liu, Z. Zhu, N. Iwamoto, Y. Peng, Z. Li, Y. Zhou, E. Bozkurt, and B. Zheng, "Beat: A large-scale semantic and emotional multi-modal dataset for conversational gestures synthesis," 2022.